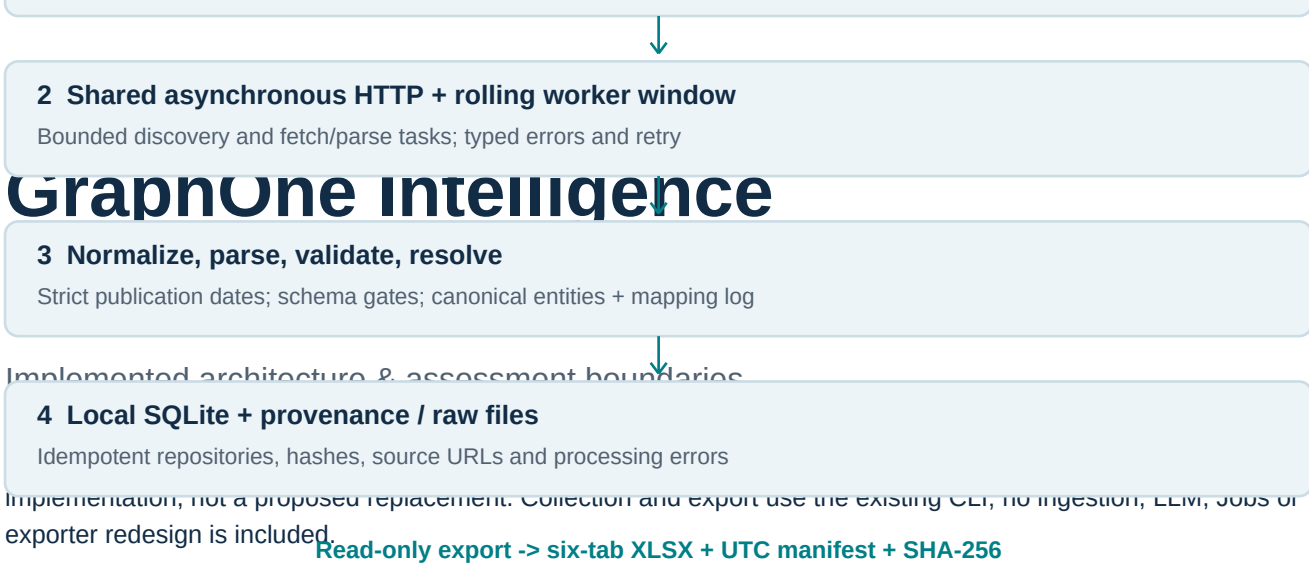




2 Shared asynchronous HTTP + rolling worker window

Bounded discovery and fetch/parse tasks; typed errors and retry

GraphOne Intelligence

3 Normalize, parse, validate, resolve

Strict publication dates; schema gates; canonical entities + mapping log

Implemented architecture & assessment boundaries

4 Local SQLite + provenance / raw files

Idempotent repositories, hashes, source URLs and processing errors

Implementation, not a proposed replacement. Collection and export use the existing CLI, no ingestion, LLM, jobs or exporter redesign is included.

Read-only export -> six-tab XLSX + UTC manifest + SHA-256

Trust boundary: source evidence before record counts

Only source-returned records that pass the implemented validators enter the database. Missing optional fields stay null; targets are not permission to invent records. Jobs and News require explicit publication/posting timestamps within the inclusive interval **[UTC now - 24 hours, UTC now]**, with zero future allowance. Export checks the interval again at one common timestamp.

Independent, implemented LLM component: Gemini Flash -> Groq-hosted Llama -> DeepSeek fallback, configured by provider/model environment variables. Current structured-data CLI verticals do **not** call this component. No live LLM execution is claimed for this submission.

Code map: src/main.py; src/config/sources.py; src/crawlers/; src/pipeline/; src/validation/; src/resolution/; src/storage/; src/export/.

01 / Ingestion & evidence

Source registry and actual adapters

Vertical	Enabled sources / execution
Startups	YC public company-directory Algolia search; AI tags; batch-partitioned pagination.
Products	Hugging Face Spaces, OpenRouter models, Hugging Face models; cursor paging and cross-source artifact deduplication.
Research Papers	arXiv Atom and OpenAlex API; explicit GitHub links only, with API enrichment when available.
Jobs (five)	RemoteOK, Working Nomads, HN Who Is Hiring, Wellfound, BuiltIn. JSON APIs/comments, sitemap JSON-LD, or public first-page job links.
News (five)	Hacker News AI, TechCrunch AI, The Verge AI, MIT Technology Review AI, The Decoder. Algolia discovery or RSS/Atom; destination article extraction.

Normalization and parsing

Adapters emit DiscoveredUrl / ParsedRecord with FetchResult provenance. Structured JSON, Atom/RSS and JobPosting JSON-LD are preferred. Pydantic schemas validate fields and HTTP URLs. Jobs retain the exact source date field/value and raw record; naive and date-only posting times are rejected. HN news uses the destination article publication date, not a recent HN submission as a substitute.

News requires extractable full text (minimum 100 characters) through trafilatura with newspaper fallback. Stale/future, invalid, inaccessible or unextractable candidates are rejected and accounted for; no replacement records are manufactured.

Entity resolution

Startups, Products and Jobs call the deterministic resolver: normalized exact match -> stored alias -> high-threshold fuzzy match (92) -> create a source-named entity, or unresolved for unusable names. Canonical IDs and mapping decisions are persisted. A resolver failure leaves a null link. Similar names are not proof of shared identity; fuzzy decisions remain reviewable.

Provenance: what is actually retained

RawDocument links retain source/fetch URLs, content hashes, HTTP status and timestamps. Jobs archive raw fetched text in local content-hash files and preserve per-item date evidence. News archives extracted article text. Startups, Products and Research retain provenance/hash metadata but **do not archive complete raw response bytes**. The workbook includes provenance joins, mapping history and available News full text; local raw-file pointers are not public download links.

02 / Reliability & persistence

LLM orchestration: implemented, not invoked by this CLI run

Gemini Flash -> Groq Llama -> DeepSeek: provider adapters share the HTTP error taxonomy.

LLM_PROVIDER_ORDER defaults to gemini,groq,deepseek; model IDs and keys must be supplied explicitly. Each provider has a bounded retry budget and validates output against a Pydantic schema. Exhausted providers fall through; total exhaustion raises ProviderUnavailableError. Exact duplicate result objects are removed without inventing or merging fields.

413: deterministic pre-chunking uses a character/token estimate. PayloadTooLargeError is never retried unchanged. Recovery halves the payload recursively, bounded by LLM_MAX_SPLIT_DEPTH (default 8) and a one-character stop. Irreducible payloads fail explicitly. Attempt metadata is recorded in LLMRequest, including latency, retry count and fallback use.

429, transient failures and source-side blocks

Shared retry_async applies bounded exponential backoff with full jitter. Retry-After supports seconds or an HTTP-date and is a minimum cooldown. A server hint exceeding the configured delay budget aborts instead of retrying early. HTTP 429 and GitHub rate-limit-flavored 403 are rate limits; other 401/403 blocks are not bypassed. Network/timeouts use bounded retry; failures remain visible in logs and processing_errors.

Anti-bot / JS-heavy strategy: prefer authorized public APIs, feeds and structured server-rendered pages. Wellfound may return an access block; BuiltIn may omit timezone-qualified dates. Both may honestly supply zero. Playwright is a dependency, **not an implemented browser fallback**. No CAPTCHA solving or credential bypass is implemented. Robots compliance requires operator review; automated robots.txt enforcement is not implemented.

Bounded concurrency, not bounded total dataset memory

The worker scheduler keeps at most $C = \text{MAX_CONCURRENCY} / \text{--workers}$ outstanding fetch-and-parse tasks. It waits for a completed slot before requesting more discovery items; there is no unbounded waiting-task queue. Discovery closes on limits or failures; child tasks are cancelled and awaited. Per-run record lists, raw-body references, URL sets and resolver/GitHub caches still grow with the run.

Local persistence and export boundary

SQLAlchemy async repositories support SQLite and PostgreSQL. Assessment collection uses **.local/submission.db** and local raw storage; DB files and secrets are ignored and excluded from Git. Unique constraints and idempotent upserts prevent duplicate inserts. Use one process at a time against this local submission DB, and export only after collection stops.

The existing **--export --output submission/graphphone_final.xlsx** command reads the DB without collection or schema creation. It writes exactly Startups, Products, Research Papers, Jobs, News and Entity Mapping Log. Literal spreadsheet cells prevent source text becoming formulas. Long fields use continuation columns, not extra tabs. Atomic XLSX save is followed by a manifest with cutoff, counts, freshness exclusions and SHA-256.

03 / Scale & assessment limits

Theoretical 500k+ route without changing business rules

Infrastructure-only scale-up is the supported no-code path: use a larger-memory/CPU host and faster storage; configure DATABASE_URL for PostgreSQL and tune connection pools and bounded --workers within source limits. Existing extraction, validation, strict freshness, resolution and idempotent persistence rules remain unchanged. This can increase available capacity but **does not prove 500k+ throughput or completion**. Size raw payloads, process memory, DB indexes and export memory before selecting infrastructure.

Scale lever	Implemented boundary / qualification
Larger host + PostgreSQL	Configurable today. SQLite serial writes are unsuitable as a shared high-throughput worker store. PostgreSQL load and contention were not benchmarked here.
Concurrency tuning	Rolling task backpressure exists. More workers do not bound all_records, URL sets, raw bodies, or resolver state.
Repeated / larger runs	Source pagination ceilings, returned availability and API quotas remain authoritative. A fresh 24h window cannot be enlarged to obtain 1,000 records.
500k+ XLSX	Excel has a per-sheet row limit; the exporter retains the workbook in memory. No end-to-end 500k export benchmark is claimed.
Horizontal scale-out	NOT turnkey today. Durable dispatch, leases/checkpoints, shared raw storage, bounded batch persistence and cross-worker resolver consistency require implementation and validation.

What must not be advertised as existing capability

Redis/CrawlJob models and S3 settings are scaffolding, not an active distributed queue or S3 backend. Setting RAW_STORAGE_BACKEND=s3 does not move raw files. Merely adding replicas is not a safe no-code distributed pipeline. The requested blanket claim that 500k+ is achieved by infrastructure changes alone is therefore **unverified**; the scale-up scenario above is theoretical and the distributed design is future work.

Assessment deliverables and evidence

The final workbook manifest is the authority for row counts and export-time freshness. Empty/short Jobs or News tabs remain valid representations of source limitations, not claims that 1,000 was achieved. Registered-source count differs from successful-source row count. Final collection evidence records the five adapters attempted for each time-sensitive vertical and actual failures.

Known gaps: no live LLM credentials/calls; no live Google Sheets publication (XLSX is the delivered format); incomplete raw-byte archival for three verticals; no browser fallback; no production distributed/load proof. The original assessment PDF was not supplied, so unknown column-level clauses cannot be certified. PR #4 remains open and unmerged.